Where Does Test-Time Reasoning Help a Small Vision-Language Model? A Clean-Year Study of Code Sandboxes, Agentic Correction, Learned Verification and Retrieval on JEE Advanced

Arpit Kumar (12340350) Saurav Gupta (12341940)
Team Erdős, Machine Learning course project, IIT Bhilai
arpitk@iitbhilai.ac.in, sauravg@iitbhilai.ac.in

***Abstract*—According to the mmJEE-Eval benchmark, open vision-language models (VLMs) answer only a small share of JEE Advanced questions correctly, and although they notice an error in their own reasoning in up to 73% of cases, they repair it in only 1.1–5.2%. We test how much of this gap can be closed at inference time without training the model. A frozen 4-bit Qwen3-VL-8B, running on a single 12 GB GPU, is combined with four modules: a Python sandbox for exact calculation, a critic from a different model family that marks the first wrong step, a learned verifier that chooses the final answer, and retrieval of solved examples. Since 2025, our first test year, influenced our design and may have been seen by the model, we used a second test year, JEE Advanced 2026 (204 questions, with the 102 Hindi versions cropped by us), fixed an improved system using 2019–2024 data only, and ran it on 2026 a single time. The framework scores 58.8% on 2026, compared with 44.3% for a single answer, and is +6.4 points above majority voting with the same compute (95% CI [+1.5, +10.8], p=0.011). Between 14 and 19% of the errors found by the critic end up corrected. We tested eight more ideas with predefined pass rules. None passed. We separated the gain into contributions from the candidate pool and the verifier. The part that comes from the larger and more varied pool of candidate answers is similar in both years (+7.4 [+0.0, +15.3] and +6.9 [+2.0, +11.8] points), whereas the part contributed by the learned selector drops from +6.3 [+1.6, +11.1] to −0.5 [−2.9, +2.5] points, even though its overall AUC increases. Questions requiring diagrams remain the weakest group (30.3% against 72.5% for text-only questions).**

I. Introduction

Recent vision-language models score well on common multimodal benchmarks, but it is unclear how much of this reflects scientific reasoning and how much reflects familiarity with the question style. mmJEE-Eval [1] was built to study this question. It contains 1,460 questions from India's JEE Advanced exam (2019–2025) in English and Hindi, and each question is given to the model as an image. Three findings from mmJEE-Eval motivated our work. Open models solve 37–45% of the held out 2025 questions, against 77–84% for the best closed models. Numerical questions, where an exact calculation is needed, account for 44.7% of the benchmark. Finally, models detect errors in their own reasoning in 21–73% of cases but manage to fix only 1.1–5.2% of them. The benchmark paper reports these weaknesses without attempting to reduce them.

We investigate whether *test-time* methods can narrow this gap without changing the model's weights, for a small open model on ordinary lab hardware. Fig. 3 shows the four modules. The *code sandbox* handles calculations: the model writes each calculation as Python, which we execute in a restricted process [2], [3]. The second is *agentic correction*, where a model from another family identifies the first wrong step and the solver rewrites the solution from that step onwards [5], [6]. The third is a *learned verifier* that gives every candidate answer a score and returns the highest-scoring one [8], [10]. The fourth *retrieves* verified solutions of earlier problems that use the same method [11].

To avoid overstating the gains, we used three evaluation controls. We compare against majority voting at matched compute [4] because these methods use more tokens than a single answer. We selected the method settings without using the test questions. To reduce possible contamination from earlier exams, we used an exam held after the model's release. Fig. 1 summarizes the development and testing sequence. After a pilot and a full run on 2025 (Phase 2), the final test was moved to the 2026 exam, all rules were fixed before it was run, and every idea tried afterwards is reported, including those that failed.

Contributions.

- A framework around a frozen 4-bit 8B VLM that needs no training of the model and fits on one 12 GB GPU. On the clean exam year it raises accuracy from 44.3% to 58.8% and is +6.4 points above majority voting at matched compute (Sec. VII).
- A clean bilingual test set for JEE advanced 2026. We cropped the 102 Hindi questions from the official papers ourselves and matched each to its English version (Sec. III).
- An evaluation protocol with the benchmark's own scoring code, compute-matched baselines, confidence intervals

that account for English–Hindi pairs, a frozen configuration and pass rules written before each experiment.
- A correction loop that fixes 14–19% of the errors found by its critic. The base paper reports 1.1–5.2% for single-pass self-correction (Sec. VII-A).

II. Related work

Benchmarks. mmJEE-Eval [1] evaluates 17 models on JEE questions given only as images, and it also includes a contamination test and a study of single-pass self-correction. JEEBench [14] (text) and OlympiadBench [15] (bilingual olympiad problems) are related benchmarks. **Tool use.** Program-of-thought prompting [2] and PAL [3] use an interpreter for arithmetic. **Sampling and voting.** Self-consistency [4] samples several chains of thought and takes a majority vote. **Self-correction.** Self-Refine [5] and CRITIC [6] alternate between critique and revision. Huang et al [7] found that models often cannot correct themselves when no outside signal is available. **Verifiers.** Outcome and process verifiers [8], [9] rank sampled solutions and are usually large trained models. **Retrieval.** Retrieval-augmented generation [11] supplies the model with external context.

III. Data and evaluation protocol

Benchmark. An mmJEE-Eval item is a single image holding the question, its options and any figure, together with metadata (subject, question type, year, language and the gold answer). There are four question types (numerical, single-correct MCQ, multiple-correct MCQ and matching), and 30.3% of the questions cannot be answered without the diagram.

Splits. We split the data by exam year (Table I). The years 2019–2024 (1,270 questions) are used for training and development, and 2024 serves as the dev year. The 2025 exam (190 questions) was our test year in Phase 2. It was later used for the pilot, it influenced several design decisions and the model may have seen it, so every 2025 number reported after Phase 2 is marked as post-hoc.

Clean test year. JEE Advanced 2026 took place after Qwen3-VL was released. The dataset has its 102 English questions, and we added the 102 Hindi versions ourselves. Each Hindi question was cropped from the official Hindi paper, matched to its English twin by paper, subject and question number, and given the answer from the English key. An automatic check confirmed every pairing, and 14 crops, among them all size outliers, were inspected by hand. While doing this we found and fixed three bugs in our parser; in one of them, the Hindi word for "part" inside a physics question had been read as a section heading. Fig. 2 shows one pair. Because our crops differ from the dataset images, we report results separately for each language.

Scoring. The functions `extract_answer` and `is_answer_correct` are taken from the base paper without changes, and a unit test confirms that every gold answer is scored as correct. Matching questions are scored in the same way as single correct MCQs. For rows with several samples, Pass@1 is the average accuracy over the samples. For rows that select one answer, we report the accuracy of that answer.

TABLE I
Data splits (questions; each year has English and Hindi twins).

Years	Role	Qs	Used for
2019–2024	train / dev	1,270	verifier data, knowledge base; 2024 = dev year
2025	Phase-2 test	190	Phase-2 results; post-hoc afterwards
2026	clean test	204	final results (Hindi built by us)

Compute matching. Self-consistency (SC) is a majority vote over plain chain-of-thought (CoT) answers. We draw 20 CoT samples per question and, for each system, use the smallest number of them whose generated tokens add up to at least the system's own token count.

Leakage control. Test questions and their twins never appear in a training pool. We removed knowledge-base entries similar to any 2025 or 2026 question, and no 2025 or 2026 data was used when the final system was frozen.

IV. Proposed method

A. Modules

The full prompts are reproduced in Appendix A. **A. Code sandbox (program of thought).** The solver writes numbered steps, with calculations placed in `python` blocks. Each block is executed in a separate subprocess (`python -I`) after an AST check that admits math, NumPy, SymPy and SciPy and rejects any file or network access, with limits on time, memory and output size. The output is inserted into the solution, after which generation continues. Before a block runs, the earlier blocks of the same solution are executed again so that their variables are available. When the solution ends without a `\boxed{}` answer, a plain CoT answer is used instead.

B. Agentic correction. The critic, InternVL3.5-8B, comes from a different model family than the solver. It reads a solution and either names the first wrong step or replies `NO_ERROR`. The same solver acts as the corrector, keeping the earlier steps and reworking the solution from the flagged step. At most three rounds are run, and the loop ends early if the critic accepts the solution or the answer stays the same for two rounds. If a critic reply is cut off, we append a short cue asking for the decision and let the critic continue greedily for a few tokens (*verdict forcing*).

C. Learned verifier. We label the 2019–2024 sampled answers using the scorer, without manual annotation (in Phase 2, 10,160 answers, of which 45.5% were correct). The features are hand crafted: length, number of steps, whether the code ran, truncation, hedging words, format, agreement with the other answers in the pool and in each half of it, critic flags, number of rounds and whether the answer changed. In Phase 2 a sentence embedding was also used. The verifier scores all

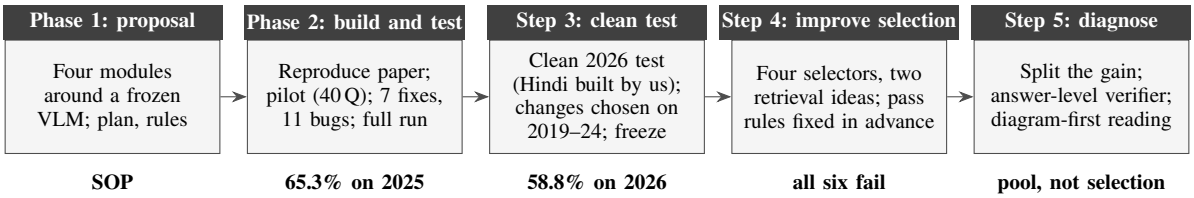

Fig. 1. Study design. Phase 2 gave a positive result on 2025, but 2025 had influenced the design, so the final test was run on 2026. On 2026 the learned selector no longer helped, which led to Step 4 (new selectors and retrieval variants, each with a fixed pass rule). None of them passed, and Step 5 then measured which parts of the system the gain comes from. Steps 4 and 5 took place after the freeze, so their results on the test years are marked post-hoc.

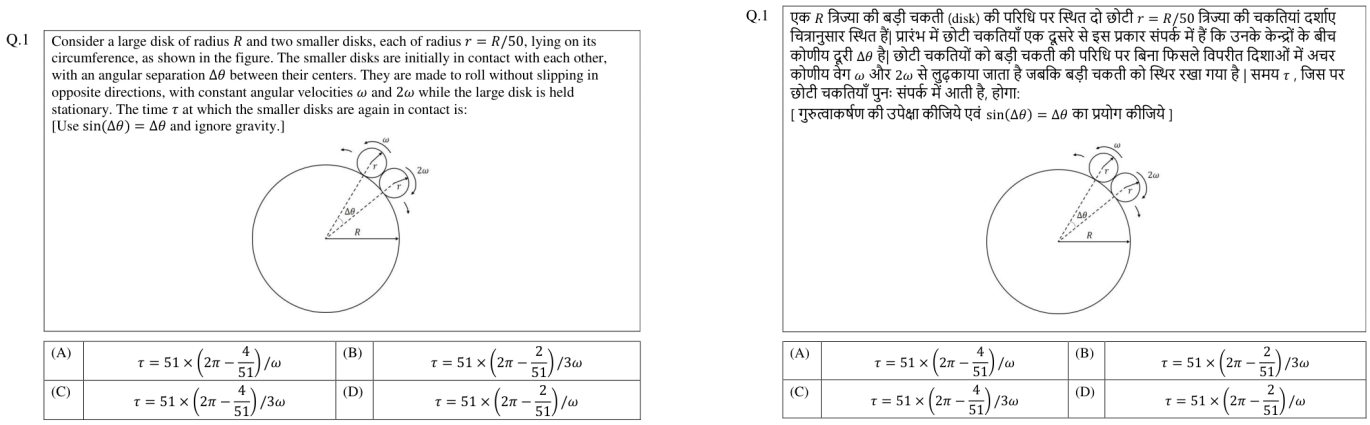

Fig. 2. A question from the clean year (JEE Advanced 2026, Paper 1, Physics Q1). Left: the English image from the dataset. Right: the Hindi version we cropped from the official paper. The model receives only the image, so it has to read the text, formulas, options and diagram itself.

16 candidates (the 8 original solutions and their 8 corrected versions) and returns the answer with the highest score.

D. Retrieval. The model first writes a one-line tag naming the topic and the method. We embed this tag with bge [13], search with FAISS [12] and take at most two verified worked solutions of the same subject from earlier years. We include these examples only when their similarity exceeds a threshold τ, excluding weak matches.

B. From the Phase-2 system to the final system

We chose every change below using only 2019–2024 data, with a rule set before testing: a change is kept only if it raises accuracy on the 2024 dev year and the lower end of its 95% CI is above zero (Table II).

- *Transcript.* The model first writes the question out in English, translating it if it is in Hindi. The solver then gets this text along with the image.
- *Continuation.* If an answer runs out of tokens, we let it go on once for up to 4,096 more tokens. We do this for every row, the baselines too so the comparison stays fair.
- *Hybrid retrieval.* The search uses the problem text as well as the tag, JEEBench 2016–2018 is added to the knowledge base, and τ is set with leave-one-year-out (LOYO) tests. More questions now get examples (8.4% $\rightarrow$ 11.7% under LOYO), the examples are as good as before, and dev accuracy stays the same.
- *Ranking verifier.* An XGBoost model that ranks the answers of a pool against each other, chosen by LOYO on 2019–2024. It is trained on pools that also hold corrected answers, just like the test pools. Under LOYO it picks a right answer in 62.0% of pools, against 53.7% for majority voting; the best possible (oracle) is 78.5%.
- *Correction gate.* A corrected answer replaces the original only if the verifier's score goes up by more than $\delta{=}-0.08$, with δ picked on other years. The gate has its own row in the results, but the final answer is still the verifier's pick from the full pool.

TABLE II
DEV A/B TESTS ON 2024 (204 QUESTIONS) THAT DECIDED THE FREEZE. RULE FIXED IN ADVANCE: INCLUDE IFF $\Delta > 0$ AND THE CI LOWER BOUND > 0.

Arm	Base	Arm	Δ [95% CI]
Transcript block (PoT)	50.2	55.1	+4.9 [+1.8, +8.2]
Continuation (PoT)	50.2	52.6	+2.3 [+1.0, +3.7]
Continuation (CoT)	48.5	54.9	+6.4 [+2.5, +10.8]
Retrieval, Phase-2 recipe	50.2	50.4	+0.2 [−2.3, +2.9]
Retrieval, hybrid + JEEBench	50.2	50.6	+0.3 [−2.1, +3.0]

We froze the configuration and committed it to the repository before the first 2026 run.

V. EXPERIMENTAL SETUP

Qwen3-VL-8B-Instruct [17] is the solver and also writes the transcript, and InternVL3.5-8B [18] is the critic. Both are frozen and quantised to 4 bits with AWQ. The base paper's model, Qwen2.5-VL-7B, is used for the reproduction and for

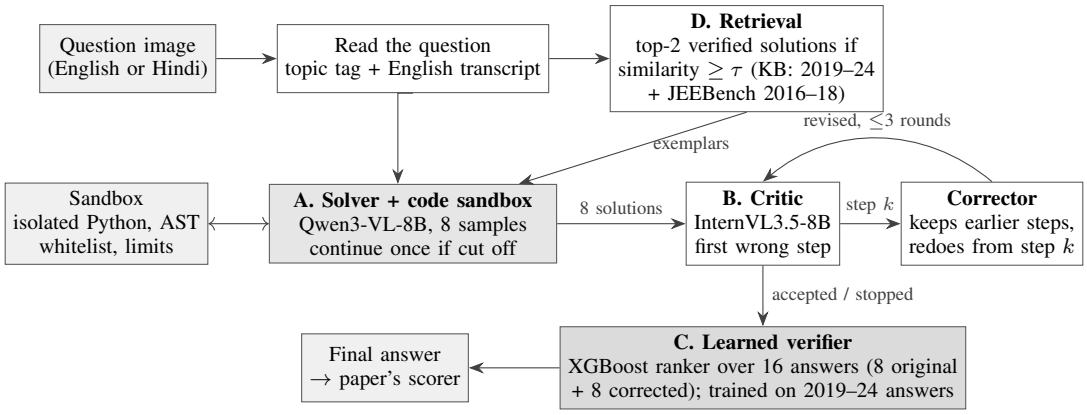

Fig. 3. The framework around the frozen VLM. The verifier (C) is the only trained part, and it is trained on automatically labelled answers from 2019–2024.

one bridge experiment. Each lab PC has one NVIDIA RTX 3060 (12 GB) and 15 GB of RAM, and the models are served with vLLM [16] in offline batch mode, one model at a time.

All rows use temperature 0.7, top-p 0.8, top-k 20 and a limit of 4,096 tokens per call. We draw 20 CoT samples per question and 8 samples for PoT and correction. We cache each model call in SQLite using fixed seeds, allowing interrupted runs to resume reproducibly.

VI. Phase 2: building and testing on 2025

Setup check. We first compared our evaluation code with a published number. Using the paper's prompts and scorer, Qwen2.5-VL-7B obtains 13.8±1.9% over 10 runs (CI 10.7–17.3), close to the 12.4±1.9% given in the paper. Qwen3-VL-8B, our solver, obtains 43.4% with one answer (16 runs) and 53.7% with a majority vote over 16 answers. Hindi questions are 10 points below English ones and diagram questions 18 points below text-only ones, and 40% of the answers reach the 4,096-token limit.

Pilot. We then ran all modules end to end on 40 test questions. Accuracy went up with each module (29.5% for a single answer, 41.2% with the code sandbox and 57.5% for the full framework), and the full framework was 22.5 points above matched SC (CI [7.5, 37.5]). The 40-question pilot gave a rough accuracy estimate but was more useful for identifying implementation problems. We read 1,520 solution traces and 640 correction chains and found seven problems in the method (Table III). Before the full GPU runs, we reviewed the code again and found 11 additional bugs, for example a crash in the verifier's cross-validation, features that were computed differently in training and at test time, and confidence intervals that ignored the link between twin questions. Each fix is covered by a regression test.

Full run on 2025. Table IV and Fig. 4 show the results as modules are added one by one. The full framework reaches 65.3%, against 43.2% for one answer, and is 12.1 points above matched SC (CI [4.2, 20.5], McNemar p=0.001). There are 35 questions that only the framework solves and 12 that only SC solves. With the code sandbox, accuracy on numerical questions goes from 40.2% to 50.4%, and overall accuracy rises by 5.0 points (p=0.012). The verifier gives the largest single increase (+11.8, CI [6.8, 16.6]), while retrieval adds 2.1 points. A single CoT sample per question scores 43.9% on 2019–2024 and 42.6% on 2025, which gives no indication that the model had memorised 2025.

Open questions after Phase 2. Three findings from Phase 2 informed our later experiments. First, on the 2025 pool the Phase-2 verifier picked 65.3%, almost the same as a plain majority vote over the same 16 answers (65.8%, oracle 85.3%). Most of the gain therefore came from the variety of the pool and little from the choice among its answers. Second, retrieval was used for only 30 of the 190 questions. Third, 2025 was

TABLE III
Problems found in the pilot traces and their fixes.

Problem	Evidence	Fix and effect
Code state lost	47 `NameErrors`; each block ran in a fresh process	Replay earlier blocks first; 0 errors
Library blocked	86 blocks needed SciPy	SciPy allowed (no file I/O); 0 blocked
No critic verdict	76 replies cut off while re-solving	Short check + verdict forcing: 1 of 501 unparsed; detection 35%→56%
Many fallbacks	480 of 539 fallbacks were long reasoning loops	Forcing an answer tested: 21.9% vs. 30.0% ⇒ rejected
SC too small	system used ≈18 answers' tokens, 16 drawn	20 CoT samples per question
Selector = vote	dev pick equal to majority	reported; 8× more training data
Batch aborted	one 12,303-token prompt	retry request by request

TABLE IV
Phase-2 framework on JEE Advanced 2025 (190 questions; the Phase-2 test year). Rows add one module at a time from the code sandbox. Accuracy in %, 95% cluster-bootstrap CI; tokens are generated tokens per question.

Configuration	Acc.	Num.	95% CI	Tok./q
Baseline (single-pass CoT)	43.2	40.2	37.0–49.6	2.7k
Self-consistency (matched, n=20)	53.2	51.2	44.2–62.6	54.2k
+ Code sandbox (PoT)	48.2	50.4	41.4–54.9	3.5k
+ Correction, same-model critic	51.2	55.1	44.6–58.0	6.1k
+ Correction, cross-model critic	51.3	53.0	44.5–58.1	6.4k
+ Learned verifier	63.2	66.7	55.3–71.6	51.5k
+ Retrieval = full framework	**65.3**	69.0	57.4–73.7	54.1k

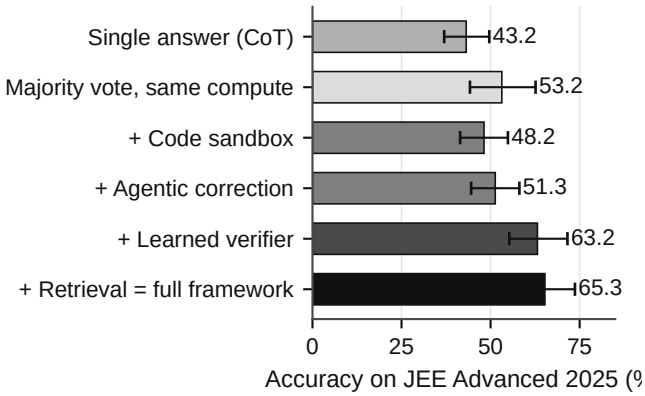

Fig. 4. Phase-2 framework on 2025, adding one module at a time (95% CIs).

TABLE V
PAIRED COMPARISONS (CLUSTER BOOTSTRAP; MCNEMAR COUNTS QUESTIONS ONLY a / ONLY b GETS RIGHT; IT IS MEANINGFUL FOR SINGLE-ANSWER ROWS ONLY, AS PASS@1 ROWS ARE ROUNDED PER QUESTION). †POST-HOC.

Comparison (a vs. b)	Δ [95% CI]	P($\Delta \leq 0$)	a/b	p
2026 final vs. Phase-2 framework	+3.4 [−1.5, +8.8]	0.113	16/9	0.230
2026 final vs. Phase-2 PoT	+6.8 [+3.6, +10.0]	0.000	27/3	<0.001
2026 CoT with continuation vs. CoT	+3.0 [+1.7, +4.5]	0.000	6/3	0.508
2026 gated vs. ungated correction	+0.6 [−0.4, +1.5]	0.122	4/1	0.375
2026 verifier pick vs. mean PoT answer	+9.0 [+5.8, +12.4]	0.000	17/0	<0.001
2025† final vs. Phase-2 framework	+6.8 [+1.1, +12.1]	0.012	21/8	0.024
2025† final vs. Phase-2 PoT	+8.1 [+4.8, +11.6]	0.000	30/7	<0.001

no longer a clean test year (Sec. III).

VII. RESULTS AND ANALYSIS ON THE CLEAN 2026 TEST

Fig. 5 shows the results for both frameworks as modules are added. The final framework reaches **58.8%**, compared with 44.3% for one answer, an increase of +14.6 points. It generates about 80k tokens per question, which buys 18 samples for matched SC, and SC with these samples reaches 52.5%. The framework is +6.4 points higher (CI [+1.5, +10.8]); it solves 18 questions that SC gets wrong, while SC solves 5 that the framework gets wrong. The Phase-2 framework, run again without changes, reaches 55.4% and is +6.4 points above its own matched SC (p=0.015). The 2026 results therefore support our main claim: the framework outperforms majority voting at matched compute. In Fig. 6, every row of the framework lies above the SC curve at the corresponding cost.

Final vs. Phase-2 framework. The paired tests are in Table V. The final solver performs better on the PoT comparison. Its PoT answers are +6.8 points better than those of the Phase-2 system (CI [+3.6, +10.0]), and continuation on its own adds +3.0 points to CoT. For the complete frameworks the difference is +3.4 points (CI [−1.5, +8.8], p=0.230), which is not significant with 204 questions. On the post-hoc 2025 set the same comparison gives +6.8 (CI [+1.1, +12.1]). The verifier accounts for most of the difference between the two years (Sec. VII-B).

TABLE VI
CORRECTION LOOP, PAPER-STYLE METACOGNITION (%). EP: WRONG SOLUTIONS FLAGGED IN ROUND 1; EC: ROUND-1 FIXES AMONG DETECTED; FA: CORRECT SOLUTIONS FLAGGED. W→R / R→W: ANSWER FLIPS AFTER ALL ROUNDS.

Run	Chains	Acc.	EP	EC	FA	W→R	R→W
2026 Ph. 2, PoT + corr.	1632	43.0→46.8	67.5	9.7	23.6	76	15
2026 Ph. 2, full	1632	44.7→48.8	69.0	10.8	25.2	83	16
2026 final	1632	49.8→52.0	52.9	13.2	19.9	68	33
2025† final	1520	56.2→57.9	40.3	13.1	23.3	46	21
2025 Ph. 2, PoT + corr.	1520	48.2→51.3	59.3	13.3	25.7	87	39
2025 Ph. 2, full	1520	50.9→54.7	60.7	15.7	28.6	87	29

TABLE VII
PRIMARY VERIFIER ON ITS OWN 16-ANSWER TEST POOL: AUC AND ACCURACY OF THE PICKED ANSWER VS. POOL BASELINES (%).

Pool	Verifier	AUC	Pick	Maj.	Rand.	Oracle
2026 final	XGBoost ranker	0.902	**58.8**	59.3	50.9	79.4
2026 Phase-2	LogReg, all feat.	0.913	**55.4**	53.9	46.7	76.5
2025† final	XGBoost ranker	0.872	**72.1**	65.8	57.1	87.4
2025 Phase-2	LogReg, all feat.	0.836	**65.3**	65.8	52.8	85.3

Results by group. Accuracy improves in every group (Fig. 7). On 2026 English questions reach 63.7% and Hindi questions 53.9%, and text-only questions reach 72.5% while diagram questions reach only 30.3%. The code sandbox and correction cannot resolve errors caused by misreading a figure. Some of the Hindi gap on 2026 could also be caused by our own crops.

A. Correction on clean data

Table VI uses the measures defined in the base paper. In round 1 the critic flags 52.9% of the wrong solutions of the final system, and 13.2% of the flagged errors are fixed in that round. Counting all rounds, 14–19% of the detected errors end with a correct answer (Fig. 8). The base paper reports 1.1–5.2% for single-pass correction. Three factors may explain the difference: step-specific feedback, a separate critic model, and numerical checks in the sandbox. The critic also flags 19.9% of the *correct* solutions, and some of these are then changed from right to wrong. Since the final solver is stronger, fewer of its solutions are wrong to begin with, and the absolute gain from correction is smaller than in Phase 2.

On 2026 the correction gate reduces the right→wrong changes from 33 to 9 and keeps most of the useful ones (wrong→right goes from 68 to 54). Accuracy per chain changes from 52.0% to 52.6% (+0.6, CI [−0.4, +1.5]). This difference is not significant, because the gate also rejects some correct fixes.

B. The verifier as a classifier and as a selector

As a classifier of individual answers, the verifier maintains a high AUC on 2026, with an AUC of 0.90 on the 2026 pool (Table VII). Judged by the answer it picks, however, it brings no gain on the clean year. It picks a right answer for 58.8% of the questions, while majority voting over the same 16 answers gives 59.3% and the oracle 79.4%. On 2025 it was

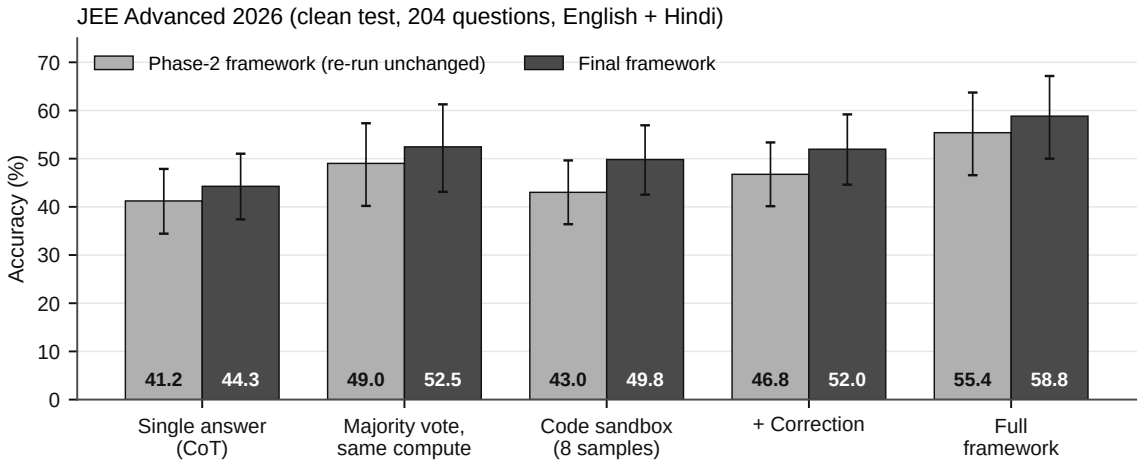

Fig. 5. Clean 2026 test: matching rows of the Phase-2 framework (re-run unchanged) and the final framework, with 95% cluster-bootstrap CIs. Majority vote uses the same number of generated tokens as the corresponding full framework.

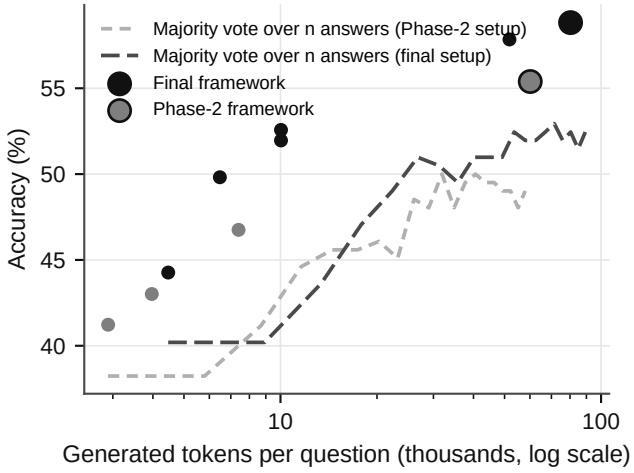

Fig. 6. 2026: accuracy vs. generated tokens per question. Dashed: majority vote over n CoT answers. Dots: rows of Fig. 5; large dots: the full frameworks.

+6.3 points above majority voting. On 2026 the improvement therefore comes from the better pool of candidates, produced by the transcript, continuation and correction, rather than from the selection step. Choosing among the 8 PoT answers alone already gives 57.8%. Before the runs we had noted one possible cause. The verifier was trained on plain PoT solutions of about 3.5k tokens, whereas the 2026 solutions of the final system are about 6.5k tokens long because of the transcript and the continuation. This explanation is tested in Sec. VIII.

C. Checks on the evaluation

Memorisation probe. When asked for the answer only (64 tokens, no reasoning), Qwen3-VL scores 10.5–15.7% on 2019–2024, 13.2% on 2025 and 15.2% on 2026 (Fig. 9). The score on the clean year is not lower than on the earlier years, so the probe shows no sign of memorisation.

Bridge to the paper's model. With Qwen2.5-VL-7B the framework gives no improvement. One answer scores 14.0%, matched SC 16.3% and the framework 15.3% (-1.1 against SC, CI $[-7.4, +5.3]$). At an accuracy of about 14% most candidates are wrong, and agreement between answers, the critic and the verifier then provide little useful information. The framework only helps when the solver is right reasonably often.

Post-hoc 2025. On 2025 the final framework reaches 72.1%, compared with 65.3% for the Phase-2 run, and it is +13.7 points above matched SC. The gaps are larger on 2025 than on 2026, which fits with some optimism on a year that influenced the design.

VIII. Can selection or retrieval be improved?

After the main test, we investigated selection and retrieval, the two weaker components. By this time we had already seen results on both test years, so all 2025 and 2026 numbers in this section and the next are post-hoc. Every choice was still made on 2019–2024 data, following a plan committed before the runs.

Error analysis. Of the 204 questions, 120 are solved. For 42 the pool holds a right answer that the selector does not pick, and for 42 the pool holds no right answer at all (Fig. 10). In the pools where the selector misses, only 17% of the candidates are right, so the right answer is clearly in the minority. Most of these errors are on diagram questions. Among them, 34.8% are selector misses and the same share never get a right answer into the pool, compared with 13.8% in each category for text-only questions. The difference in solution length does not explain the drop. The final system's solutions are longer than the training solutions in both years (7.3k and 8.3k tokens against 4.0k, with small standardised differences of 0.15 and 0.28), and yet the verifier is better than majority voting on

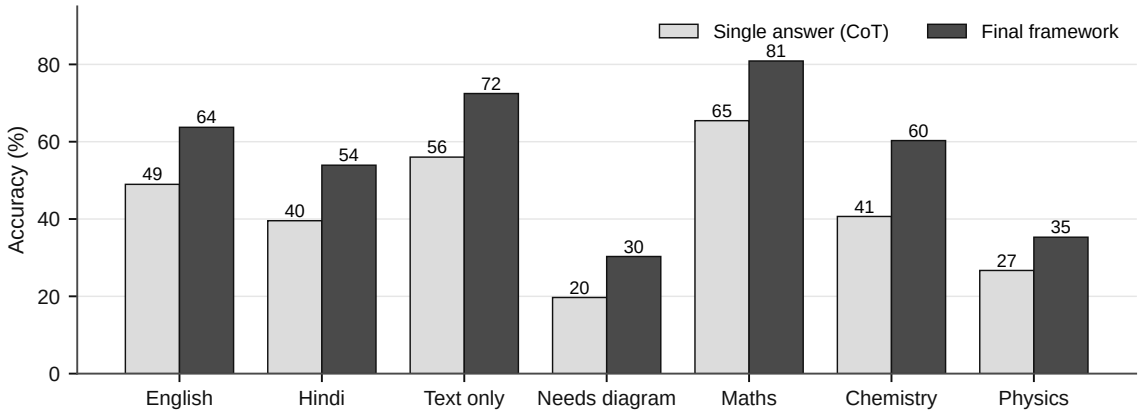

Fig. 7. 2026 accuracy by group: one CoT answer vs. the final framework.

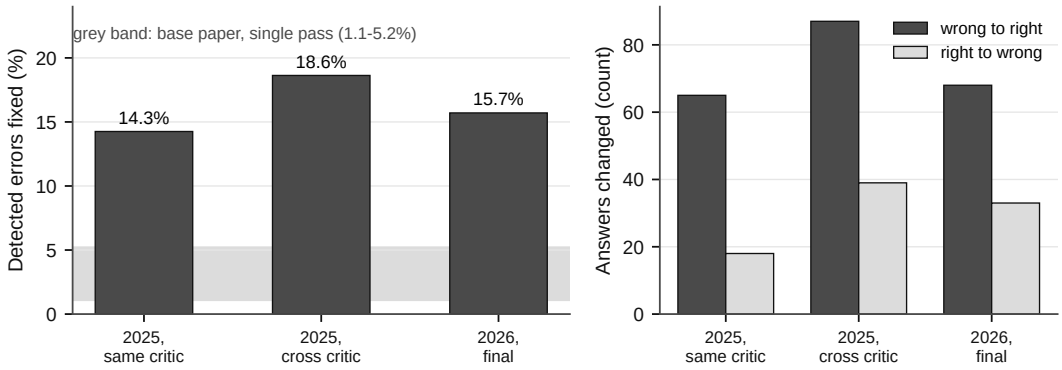

Fig. 8. Correction loop. Left: share of detected errors that are correct after up to three rounds; the grey band is the base paper's single-pass rate of 1.1–5.2%. Right: answers changed by correction, in each direction.

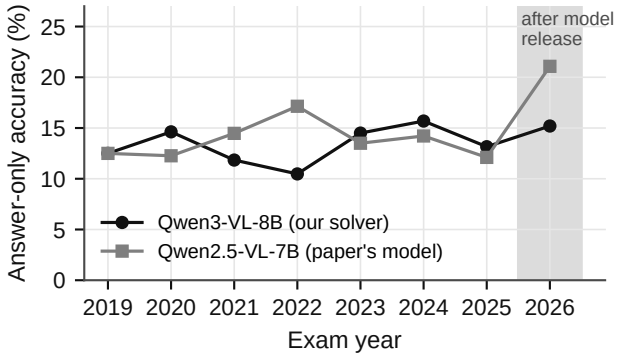

Fig. 9. Answer-only accuracy by exam year. 2026 is after both models' release.

TABLE VIII
NEW SELECTORS: PICK ACCURACY (%) UNDER LOYO ON THE 16-ANSWER TRAINING POOLS (2019–2024, 242 POOLS; NO TEST DATA). PASS RULE (FIXED IN ADVANCE): BEAT THE FROZEN RANKER ON EVERY POOL TYPE. RANKER LOYO NOISE IS ABOUT 1 POINT.

Selector	LOYO pick	Rule
Frozen ranker (final framework)	62.0	reference
Ranker without length features	59.1	fail
Fine-tuned neural verifier (ModernBERT)	57.0	fail
Ranker + neural score (stacked)	60.3	fail
Ranker with 3,184 matched training answers	61.2	fail
Answer-level verifier (Sec. IX)	57.9	fail
Majority vote	53.7	
Oracle	78.5	

2025 (+6.3 [+1.6, +11.1]) and not on 2026 (−0.5 [−2.9, +2.5]).

New selectors (Table VIII). We tried four variants. The first is the ranker with its length features removed, which tests the length explanation directly; its accuracy goes down. The second is a neural verifier, a ModernBERT-base cross-encoder [19] fine-tuned on the English transcript, the answer and the end of the solution (1,024 tokens), with one fold per training year. The neural verifier achieves a candidate AUC of 0.787 (between 0.744 and 0.817 across years), and used as a weight in the vote it is better than plain majority voting, but as a ranker it is worse than the handcrafted model. The

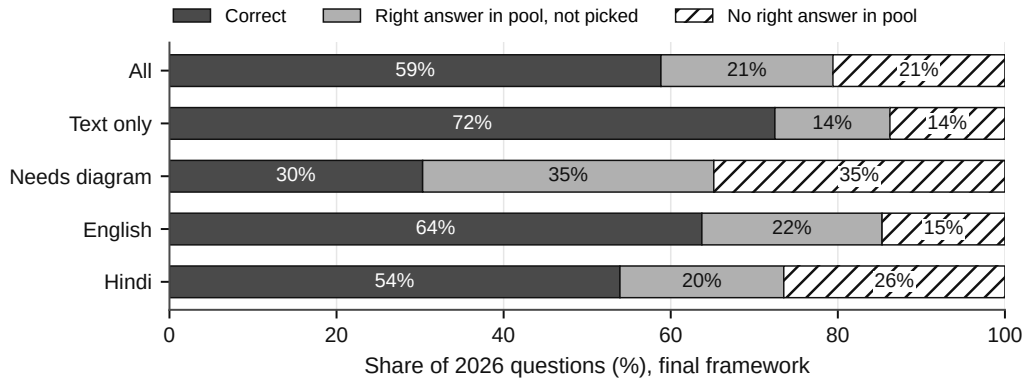

Fig. 10. Outcome of each 2026 question for the final framework: solved, right answer present in the pool but not picked, or no right answer in the pool.

TABLE IX
RETRIEVAL VARIANTS, PAIRED DIFFERENCES (POINTS, 95% CLUSTER-BOOTSTRAP CI). TOP: 2024 DEV YEAR WITH MATCHED SEEDS, 204 QUESTIONS, RAG FIRES ON 31. BOTTOM: POST-HOC 2026 POT WITH THE EXTENDED KB (†RIGHT COLUMN: QUESTIONS WHOSE RETRIEVAL CHANGED). LOYO COVERAGE AT PROXY PRECISION 0.70: EXTENDED KB 13.5% VS. CURRENT KB 15.0%.

Comparison	All	RAG fires
Solutions − no RAG	−0.2 [−1.0, +0.4]	−1.6 [−6.2, +2.5]
Cards − no RAG	+0.3 [−0.2, +0.9]	+2.0 [−1.6, +6.2]
Cards − solutions	+0.6 [−0.1, +1.3]	+3.6 [−0.8, +8.5]
Extended KB − current KB, 2026†	+0.0 [−0.4, +0.4]	+0.0 [−6.2, +7.3]

TABLE X
WHERE THE FINAL FRAMEWORK'S GAIN OVER MATCHED SELF-CONSISTENCY COMES FROM (%; DIFFERENCES IN POINTS, 95% CLUSTER-BOOTSTRAP CI). POOL GAIN: MAJORITY VOTE ON OUR POOL − MATCHED SC; SELECTION GAIN: VERIFIER PICK − MAJORITY ON THE SAME POOL. FROZEN VERIFIER AND EXISTING POOLS ONLY. MIXED POOLS HOLD BOTH RIGHT AND WRONG ANSWERS (NUMBER OF POOLS IN BRACKETS).

	2025	2026
Matched self-consistency	58.4	52.5
Majority vote on our pool	65.8	59.3
Verifier pick	72.1	58.8
Pool gain	+7.4 [+0.0, +15.3]	+6.9 [+2.0, +11.8]
Selection gain	+6.3 [+1.6, +11.1]	−0.5 [−2.9, +2.5]
Total gain	+13.7 [+5.8, +21.6]	+6.4 [+1.5, +11.3]
Candidate AUC, all	0.872	0.902
Candidate AUC, mixed pools	0.796 (129)	0.748 (121)

third adds the neural score to the ranker as one more feature. The fourth retrains all of the above with 3,184 new training answers produced in the style of the final system (398 training questions, about 30 GPU-hours), so that the training pools look like the test-time solutions. None of them meet its pass rule and as planned, none was evaluated on the test years.

Retrieval (Table IX). In these runs the retrieval arms reuse the seeds of the run without retrieval. A question that receives no examples therefore produces exactly the same output, and the effect is measured only on the questions where retrieval is used (31 of the 204 dev questions). We tested two ideas. The first replaces the worked solution with a short *method card* that states the method, when it applies, a plan, the key equations and a common mistake, and leaves out the answer. On the questions with retrieval, cards are +3.6 [−0.8, +8.5] points better than solutions ($P(\Delta \leq 0)=0.07$). The lower end of the interval is below zero, so cards were not adopted under our rule. The second idea extends the knowledge base with 207 verified OlympiadBench solutions, which are released under Apache-2.0. LOYO coverage then falls from 15.0% to 13.5%, because the new close neighbours push up the tuned threshold while being retrieved only 11 times themselves. On 2026, retrieval changes for 12 questions and the accuracy stays the same. We did not use coaching material, because of its licences and the risk of overlap with the test questions.

IX. WHERE DOES THE GAIN COME FROM?

We ran three last analyses, each with a pass rule fixed beforehand.

Pool gain and selection gain (Table X, Fig. 11). We split the gain over matched SC into two parts. The *pool gain* is how much a majority vote on our 16-answer pool beats matched SC, and the *selection gain* is how much the verifier's pick beats that vote. The pool gain is about the same in both years (+7.4 [+0.0, +15.3] and +6.9 [+2.0, +11.8] points), but the selection gain falls from +6.3 [+1.6, +11.1] to −0.5 [−2.9, +2.5]. The verifier's overall AUC is in fact higher on 2026 (0.90 against 0.87), because most of it comes from telling easy questions from hard ones. Picking only matters in pools with both right and wrong answers, and inside those pools the mean AUC falls from 0.80 to 0.75. Overall AUC therefore does not reliably measure how well the verifier selects among candidates in mixed pools.

Answer-level verifier. The ranker scores each solution on its own. We also tried a model that scores each *distinct answer* in a pool, using features such as its vote share, how many original and corrected solutions support it, whether their code ran, and the question type (5,335 answer rows). We kept the setting

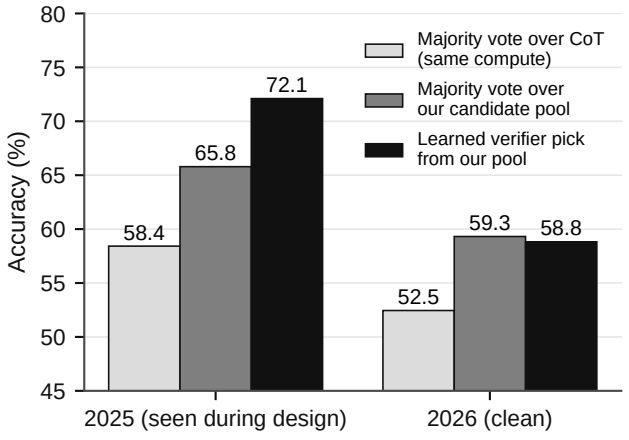

Fig. 11. Decomposition of the gain. In both years a majority vote over our pool is about 7 points above matched SC. Picking with the verifier adds a further 6.3 points on 2025 and nothing on 2026.

with the best worst-year LOYO score. It gets 57.9% (worst year 50.0%), against 62.0% (52.5%) for the ranker and 53.7% for majority voting, so it does not pass. Grouping solutions by answer beats voting, but it loses the per-solution detail that the ranker uses.

Diagram-first transcript (Fig. 12). The usual transcript describes the figure in one line. We tried a longer reading that lists every label, value with unit, arrow and axis in the figure, and solved half of each pool from it with the same seeds, so only the transcript differs. On the 2024 dev year majority@8 changes by -2.0 [-5.4, $+1.0$] points, the rule is not met, and we did not run it on 2026. Accuracy per answer drops by -3.4 [-6.9, -0.1] points, mostly on diagram questions (29.7%→24.6%), because the longer reading uses up tokens and more answers get cut off (37.3%→42.6%). It gives more distinct answers but no extra right ones (oracle@8 79.9%→78.4%). So the weakness on diagram questions is not about how the transcript is written: the 8B solver misreads the figure, or reasons wrongly about it, either way.

X. Ablation study

Fig. 5 already adds the modules one at a time. Table XI brings together the experiments in which a single component was switched on or off with everything else held fixed. The numbers are taken from Tables II, V, VIII and X and from Sec. IX.

The changes to the Solver have the clearest effect. The transcript and the continuation each help on the dev year with intervals above zero, and together they make the final PoT answers +6.8 points better than the Phase-2 ones on 2026. Choosing among the 8 PoT answers with the verifier gives a large improvement over a single PoT answer, but most of that improvement is also obtained by a plain majority vote, and on 2026 the verifier adds nothing on top of the vote. Retrieval, the correction gate and method cards all have intervals that include zero, so with the number of questions available we cannot tell whether they help. Removing the length features lowers the ranker's LOYO accuracy, and the diagram-first reading does not help on the dev year.

TABLE XI
Ablations: effect of one component with all others fixed (accuracy points, 95% CI). Dev: 2024 dev year. LOYO: pick accuracy under leave-one-year-out on 2019–2024. †post-hoc.

Component	Comparison	Data	Δ [95% CI]
Transcript	PoT with vs. without	dev	+4.9 [+1.8, +8.2]
Continuation	PoT with vs. without	dev	+2.3 [+1.0, +3.7]
Continuation	CoT with vs. without	2026	+3.0 [+1.7, +4.5]
Solver changes	final vs. Phase-2 PoT	2026	+6.8 [+3.6, +10.0]
Retrieval	hybrid retrieval vs. none	dev	+0.3 [−2.1, +3.0]
Method cards	cards vs. solutions, where used	dev	+3.6 [−0.8, +8.5]
Correction gate	gated vs. ungated chains	2026	+0.6 [−0.4, +1.5]
Verifier	pick vs. mean PoT answer	2026	+9.0 [+5.8, +12.4]
Verifier	pick vs. majority, same 16	2025†	+6.3 [+1.6, +11.1]
Verifier	pick vs. majority, same 16	2026	−0.5 [−2.9, +2.5]
Length features	ranker without vs. with	LOYO	59.1 vs. 62.0
Diagram-first reading	majority@8 with vs. without	dev	−2.0 [−5.4, +1.0]

XI. Discussion

Findings supported by the 2026 test. For both frameworks, sampling with code, correction by a second model and answer selection together beat voting over plain CoT at the same cost. The transcript and the continuation make the solver clearly stronger, and pointing the corrector to one wrong step fixes far more errors than the single-pass correction in the base paper.

Results that did not generalize. The learned selector beat majority voting under LOYO and on 2025 but not on 2026, and none of the fixes tried in Secs. VIII and IX brought this back. Retrieval has only a small effect, because few questions get examples.

Remaining failure cases. Diagram and Hindi questions are still well behind, and about two in five PoT answers are still cut off. Since most of the gain comes from candidate generation, future work should focus on increasing the number of pools containing a correct answer. For diagram questions this means reading the figure better, since better reasoning or selection cannot fix a figure that was read wrong.

XII. Limitations

We use one solver model and one clean year with 204 questions, which gives confidence intervals of about ±8 points, so differences of a few points cannot be resolved. The Hindi 2026 crops were made by us. The transcript and the knowledge-base texts are produced by the same VLM that solves the questions. Compute matching counts generated tokens only, and does not count prompt tokens or code execution. All results obtained after the freeze (Secs. VIII–X) are post-hoc for the test years. The retrieval tests on the dev year rest on only about 30 questions where retrieval is used, and the diagram-first test was run on a single dev year.

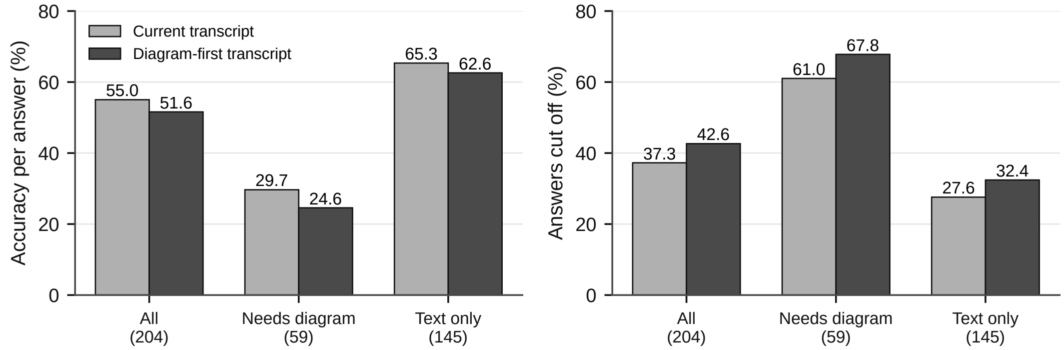

Fig. 12. Diagram-first transcript on the 2024 dev year (samples 4–7, same seeds). Left: accuracy per answer. Right: share of answers cut off at the token limit.

XIII. Conclusion and Future Work

On an exam held after the model's release, a frozen 8B VLM combined with a code sandbox, correction by a second model and answer selection improves accuracy from 44.3% to 58.8% and exceeds majority voting at the same compute by +6.4 points. The result of the Phase-2 framework also holds on this year (+6.4). The gain comes from a better pool of candidate answers and is stable across years, whereas the learned selection is fragile, and none of the eight ideas tested after the freeze changed this.

Our results suggest four areas for further investigation. (1) Diagram questions are the largest group of failures, and they need better reading of figures, for instance a stronger vision model used only to describe the diagram, or a solver that is allowed to look at parts of the image again. (2) Hindi questions would benefit from crops that match the style of the dataset and from checking the translation in the transcript. (3) Retrieval and method cards need a larger dev set before their effect can be confirmed. (4) A selector that is robust across years, probably needs features that compare solutions within a pool rather than score each one alone, and it should be judged by its accuracy within mixed pools, not by its overall AUC.

Appendix A
Prompts Given to the Models

This appendix lists the prompts that the framework sends to the two frozen models, copied from the code. The baseline prompt is the one from the base paper (its Appendix C). The Hindi variant and the prompts for the other question types follow the same pattern. Text in braces are filled in at run time. No model is trained apart from the verifier, which does not use a prompt.

a) Baseline CoT (base paper, verbatim; English, numerical question):

```
Please analyze the image carefully, reason step-by-step
and provide your answer at the end.

For this question:
- Provide a numerical value
- Round to appropriate decimal places if needed
- Format your answer in \boxed{} (e.g., \boxed{2.5} or
\boxed{42})
```

b) Code-sandbox solver instructions (appended to the baseline prompt):

````
How to solve:
- Write the solution as numbered steps, each starting on a
new line with "Step 1:", "Step 2:", and so on.
- Explain the physics/chemistry/mathematics of each step
in words.
- Do NOT do calculations in your head. Whenever a step
needs a calculation (arithmetic, algebra, solving
equations, derivatives, integrals, limits, unit
conversions, or checking each option), write a short
Python program in a ```python code block that prints the
result. You may use math, fractions, sympy and numpy.
- End the code block with ```. The code will be run and
its printed output will be given back to you in an
```output block. Then continue the solution using that
output. Never invent the output yourself.
- Use at most 3 code blocks.
- Be concise: do not re-check the same step again and
again.
- Finish with exactly one final answer in \boxed{}. Do not
use \boxed anywhere else. For a numerical answer write a
plain decimal number inside the box (no units, no
fractions, no LaTeX), rounded to two decimal places if it
is not an integer.

Required format (example):
Step 1: <which law / concept applies and the equation to
use>
```python
import sympy as sp
x = sp.symbols('x')
print(sp.solve(2*x - 6, x))
```
```output
[3]
```
Step 2: <interpret the printed result, check the options>
\boxed{<answer>}
````

c) Transcript block (final framework):

```
A transcription of the question image is given below to
help you read it. It was made automatically and may
contain errors: the image is authoritative.
Transcription:
{transcript}
```

d) Transcriber:

```
Transcribe this exam question exactly as plain text in its
original language. Write mathematical expressions in
LaTeX. Include all answer options. If there is a figure,
add one line "[Figure: <short description>]". Output only
the transcription.
```

e) Translator (Hindi transcripts):

```
Translate this JEE Advanced exam question from Hindi to
English. Keep every number, unit, LaTeX expression, option
```
```

```
label and figure note unchanged. Output only the
translation.

{text}
```

f) Concept tagger (retrieval query):

```
Look at this JEE Advanced exam question (it may be written
in English or Hindi). Do not solve it. In English,
describe what is needed to solve it, using exactly this
format on one line:
SUBJECT: <Physics/Chemistry/Mathematics> | TOPIC:
<specific topic, 2-6 words> | METHOD: <key concept or
solution technique, at most 15 words>
```

g) Exemplar header (retrieval):

```
Below are worked solutions of earlier JEE Advanced
problems that use a similar method. Use them only as
guidance on the method. They are DIFFERENT problems: do
not copy their numbers or answers.

### Example {i}
Problem:
{problem}

Worked solution:
{solution}
```

h) Critic:

```
You are a strict examiner checking a student's solution to
a JEE Advanced question. The question is in the attached
image{lang_note}.

Instructions the student was given:
{type_instruction}

Student's solution (steps are numbered):
<solution>
{solution}
</solution>

Check the steps in order: reading of the question and
figure, chosen concept and equations, algebra and
arithmetic (the ```output blocks are real program
outputs), and the final option or number. Find the FIRST
step that contains an error. Do not solve the whole
problem again. Keep your check short (at most about 250
words), then end your reply with exactly these three
lines:
VERDICT: CORRECT or ERROR
FIRST_ERROR_STEP: <step number, or NONE>
EXPLANATION: <one or two sentences: what is wrong and how
to fix it>
```

i) Critic verdict cue (verdict forcing):

```
I must stop checking now and give my decision.
VERDICT:
```

j) Corrector feedback (appended to the solver prompt):

```
A reviewer checked an earlier attempt at this question.
The steps before Step {k} were accepted and are already
written at the start of your answer. The reviewer found an
error in Step {k}:
"{explanation}"
Continue the solution from Step {k}: re-derive that step
correctly (do not repeat the error) and complete the
solution, following the same rules (Python code for every
calculation, one final \boxed{{}} answer).
```

k) Method card writer (Step 4):

```
Below is a solved JEE Advanced problem. Write a short
METHOD CARD that would help a student solve a DIFFERENT
problem of the same kind. Use exactly these five lines:
Method: <the concept or technique>
Applicable when: <conditions that signal this method>
Plan: <3-5 steps joined by ->, no numbers from this
problem>
Key equations: <the general formulas used>
Common mistake: <the trap to avoid>
Do NOT state this problem's final answer, its numerical
values or option letters. At most 150 words.

Problem:
{problem}

Worked solution:
{solution}
```

l) Diagram-first transcriber (Step 5):

```
Read this JEE Advanced exam question (it may be written in
English or Hindi) and write it out in English. Do not
solve it. Use exactly two parts:
QUESTION: the full question text and all answer options,
with mathematics in LaTeX.
FIGURE: if there is a figure, describe it as a list with
one item per line: every label and symbol, every given
value with its unit, every arrow and its direction, how
the parts are connected or placed relative to each other,
and any axes, coordinates, curves or angles. Write
"FIGURE: none" if there is no figure.
Output only these two parts.
```

m) Memorisation probe (answer only):

```
Do NOT show any reasoning, steps or calculations. Reply
with only the final answer in \boxed{}.
```

Reproducibility

The code, configurations, pre-registrations, result files and tests are available in our public repository (https://github.com/Saurav1375/erdos-mmjee-reasoner), with one branch for each phase. Every table and figure in this paper can be rebuilt from the saved result files with the scripts in the repository.

References

[1] A. Mukherjee and S. Ghosh, "mmJEE-Eval: A bilingual multimodal benchmark for evaluating scientific reasoning in vision-language models," in *Findings of IJCNLP-AACL*, 2025, pp. 2268–2290.

[2] W. Chen, X. Ma, X. Wang and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *TMLR*, 2023.

[3] L. Gao *et al.*, "PAL: Program-aided language models," in *Proc. ICML*, 2023.

[4] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Proc. ICLR*, 2023.

[5] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Proc. NeurIPS*, 2023.

[6] Z. Gou *et al.*, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. ICLR*, 2024.

[7] J. Huang *et al.*, "Large language models cannot self-correct reasoning yet," in *Proc. ICLR*, 2024.

[8] K. Cobbe *et al.*, "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[9] H. Lightman *et al.*, "Let's verify step by step," in *Proc. ICLR*, 2024.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016, pp. 785–794.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[12] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, 2019.

[13] S. Xiao *et al.*, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. SIGIR*, 2024.

[14] D. Arora, H. G. Singh and Mausam, "Have LLMs advanced enough? A challenging problem solving benchmark for large language models," in *Proc. EMNLP*, 2023.

[15] C. He *et al.*, "OlympiadBench: A challenging benchmark for promoting AGI with Olympiad-level bilingual multimodal scientific problems," in *Proc. ACL*, 2024.

[16] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[17] Qwen Team, "Qwen3-VL," model release and technical report, 2025.

[18] OpenGVLab, "InternVL3.5," model release and technical report, 2025.

[19] B. Warner *et al.*, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," arXiv:2412.13663, 2024.